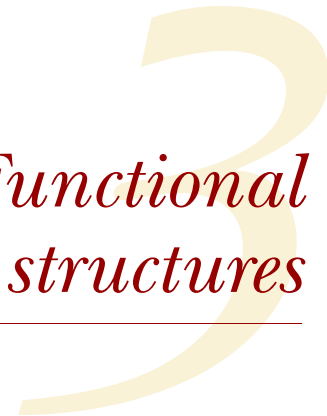




# Functional data structures

---

We said in the introduction that functional programs don't update variables or modify mutable data structures. This raises pressing questions: what sort of data structures *can* we use in functional programming, how do we define them in Scala, and how do we operate on them? In this chapter, we'll learn the concept of *functional data structures* and how to work with them. We'll use this as an opportunity to introduce how data types are defined in functional programming, learn about the related technique of *pattern matching*, and get practice writing and generalizing pure functions.

This chapter has a lot of exercises, particularly to help with this last point—writing and generalizing pure functions. Some of these exercises may be challenging. As always, consult the hints or the answers at our GitHub site (<https://github.com/fpinscala/fpinscala>; see the preface), or ask for help online if you need to.

## 3.1 Defining functional data structures

A functional data structure is (not surprisingly) operated on using only pure functions. Remember, a pure function must not change data in place or perform other side effects. *Therefore, functional data structures are by definition immutable.* For example, the empty list (written `List()` or `Nil` in Scala) is as eternal and immutable as the integer values 3 or 4. And just as evaluating  $3 + 4$  results in a new number 7 without modifying either 3 or 4, concatenating two lists together (the syntax for this is `a ++ b` for two lists `a` and `b`) yields a new list and leaves the two inputs unmodified.

Doesn't this mean we end up doing a lot of extra copying of the data? Perhaps surprisingly, the answer is no, and we'll talk about exactly why that is. But first let's examine what's probably the most ubiquitous functional data structure, the singly linked list. The definition here is identical in spirit to (though simpler than) the

List data type defined in Scala's standard library. This code listing introduces a lot of new syntax and concepts that we'll talk through in detail.

### Listing 3.1 Singly linked lists

```
package fpinscala.datastructures

sealed trait List[+A]
case object Nil extends List[Nothing]
case class Cons[+A](head: A, tail: List[A]) extends List[A]

object List {
  def sum(ints: List[Int]): Int = ints match {
    case Nil => 0
    case Cons(x,xs) => x + sum(xs)
  }

  def product(ds: List[Double]): Double = ds match {
    case Nil => 1.0
    case Cons(0.0, _) => 0.0
    case Cons(x,xs) => x * product(xs)
  }

  def apply[A](as: A*): List[A] =
    if (as.isEmpty) Nil
    else Cons(as.head, apply(as.tail: _*))
}
```

**List companion object. Contains functions for creating and working with lists.**

**List data type, parameterized on a type, A.**

**A List data constructor representing the empty list.**

**Another data constructor, representing nonempty lists. Note that tail is another List[A], which may be Nil or another Cons.**

**The sum of a list starting with  $x$  is  $x$  plus the sum of the rest of the list.**

**The sum of the empty list is 0.**

**A function that uses pattern matching to add up a list of integers.**

**Variadic function syntax.**

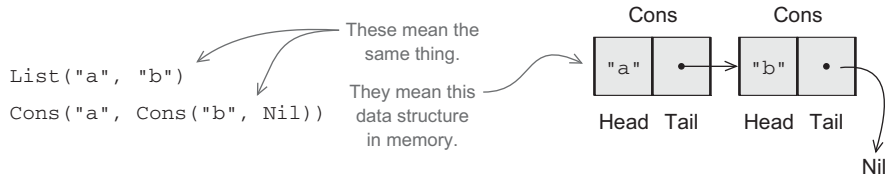
Let's look first at the definition of the data type, which begins with the keywords `sealed trait`. In general, we introduce a data type with the `trait` keyword. A trait is an abstract interface that may optionally contain implementations of some methods. Here we're declaring a trait, called `List`, with no methods on it. Adding `sealed` in front means that all implementations of the trait must be declared in this file.<sup>1</sup>

There are two such implementations, or *data constructors*, of `List` (each introduced with the keyword `case`) declared next, to represent the two possible forms a `List` can take. As the figure at the top of page 31 shows, a `List` can be empty, denoted by the data constructor `Nil`, or it can be nonempty, denoted by the data constructor `Cons` (traditionally short for *construct*). A nonempty list consists of an initial element, `head`, followed by a `List` (possibly empty) of remaining elements (the `tail`):

```
case object Nil extends List[Nothing]
case class Cons[+A](head: A, tail: List[A]) extends List[A]
```

<sup>1</sup> We could also say `abstract class` here instead of `trait`. The distinction between the two is not at all significant for our purposes right now. See section 5.3 in the Scala Language Specification (<http://mng.bz/R75t>) for more on the distinction.

## Singly linked lists



Just as functions can be polymorphic, data types can be as well, and by adding the type parameter `[+A]` after sealed trait `List` and then using that `A` parameter inside of the `Cons` data constructor, we declare the `List` data type to be polymorphic in the type of elements it contains, which means we can use this same definition for a list of `Int` elements (denoted `List[Int]`), `Double` elements (denoted `List[Double]`), `String` elements (`List[String]`), and so on (the `+` indicates that the type parameter `A` is *covariant*—see sidebar “More about variance” for more information).

A data constructor declaration gives us a function to construct that form of the data type. Here are a few examples:

```
val ex1: List[Double] = Nil
val ex2: List[Int] = Cons(1, Nil)
val ex3: List[String] = Cons("a", Cons("b", Nil))
```

The case object `Nil` lets us write `Nil` to construct an empty `List`, and the case class `Cons` lets us write `Cons(1, Nil)`, `Cons("a", Cons("b", Nil))` and so on to build singly linked lists of arbitrary lengths.<sup>2</sup> Note that because `List` is parameterized on a type, `A`, these are polymorphic functions that can be instantiated with different types for `A`. Here, `ex2` instantiates the `A` type parameter to `Int`, while `ex3` instantiates it to `String`. The `ex1` example is interesting—`Nil` is being instantiated with type `List[Double]`, which is allowed because the empty list contains no elements and can be considered a list of whatever type we want!

Each data constructor also introduces a *pattern* that can be used for *pattern matching*, as in the functions `sum` and `product`. We’ll examine pattern matching in more detail next.

### More about variance

In the declaration `trait List[+A]`, the `+` in front of the type parameter `A` is a *variance annotation* that signals that `A` is a *covariant* or “positive” parameter of `List`. This means that, for instance, `List[Dog]` is considered a subtype of `List[Animal]`, assuming `Dog` is a subtype of `Animal`. (More generally, for all types `X` and `Y`, if `X` is a

<sup>2</sup> Scala generates a default `def toString: String` method for any case class or case object, which can be convenient for debugging. You can see the output of this default `toString` implementation if you experiment with `List` values in the REPL, which uses `toString` to render the result of each expression. `Cons(1, Nil)` will be printed as the string `"Cons(1, Nil)"`, for instance. But note that the generated `toString` will be naively recursive and will cause stack overflow when printing long lists, so you may wish to provide a different implementation.